

Programming Assignment Report

Design and Analysis of Algorithms

Name: Harshit S Gowda

Registration Number: 2411021061315

Problem Name: Largest Permutation

Problem Statement:

You are given an unordered array of unique integers incrementing from. You can swap any two elements a limited number of times. Determine the largest lexicographical value array that can be created by executing no more than the limited number of swaps.

Example

arr = [1,2,3,4]

K = 1

The following arrays can be formed by swapping the with the other elements:

[2,1,3,4]

[3,2,1,4]

[4,2,3,1]

The highest value of the four (including the original) is [4,2,3,1]. If $k \geq 2$, we can swap to the highest possible value: [4,3,2,1].

Function Description

Complete the largest Permutation function in the editor below. It must return an array that represents the highest value permutation that can be formed.

Largest Permutation has the following parameter(s):

- int k: the maximum number of swaps
- int arr[n]: an array of integers

Input Format

The first line contains two space-separated integers n and k, the length of arr and the maximum swaps that can be performed. The second line contains n distinct space-separated integers from 1 to n as arr[i] where $1 \leq arr[i] \leq n$.

Constraints

$1 \leq n \leq 10^5$

$1 \leq k \leq 10^9$

Output Format

Print the lexicographically largest permutation you can make with at most k swaps.

Sample Input 0

STDIN Function

5 1 $n = 5, k = 1$

4 2 3 5 1 $arr = [4, 2, 3, 5, 1]$

Sample Output 0

5 2 3 4 1

Explanation 0

You can swap any two numbers in $[4, 2, 3, 5, 1]$ and see the largest permutation is $[5, 2, 3, 4, 1]$

Sample Input 1

3 1

2 1 3

Sample Output 1

3 1 2

Explanation 1

With 1 swap we can get $[1, 2, 3]$, $[3, 1, 2]$, and $[2, 3, 1]$. Of these, $[3, 1, 2]$ is the largest permutation.

Sample Input 2

2 1

2 1

Sample Output 2

2 1

Explanation 2

We can see that $[2, 1]$ is already the largest permutation. We don't make any swaps.

Problem Hacker Rank Link: [harshit-s-gowda/hackerrank-solutions-LargestPermutation: Solutions to HackerRank coding problems in Java.](https://www.hackerrank.com/solutions/LargestPermutation?h_r=harshit-s-gowda&h_s=solutions-LargestPermutation)

Github Submission Link: [hackerrank-solutions-Largest-Permutation/DAA_ASSIGNMENT_REPORT_HACKER_RANK_PROBLEM.pdf](https://github.com/harshit-s-gowda/hackerrank-solutions-Largest-Permutation/blob/main/DAA_ASSIGNMENT_REPORT_HACKER_RANK_PROBLEM.pdf) at main · [harshit-s-gowda/hackerrank-solutions-Largest-Permutation](https://github.com/harshit-s-gowda/hackerrank-solutions-Largest-Permutation)

Solution Steps:

Step 1: Read the input values

Read the values n (size of the array) and k (maximum number of swaps).
Then read the array elements and store them in an integer array.

Step 2: Store element positions

Create a HashMap to store each element and its current index in the array.
This helps in quickly finding the position of any number during swaps.

Step 3: Identify the largest value

Set a variable $\text{maxVal} = n$, which represents the largest possible value that should appear at the current position in the permutation.

Step 4: Traverse the array

Loop through the array from the first index to the last index while $k > 0$ (swaps are still available).

Step 5: Check if the current element is correct

If the element at the current position is already equal to maxVal , it means the correct largest number is already placed.

So move to the next position and reduce maxVal .

Step 6: Find the position of the largest value

If the current element is not the expected largest value, use the HashMap to find the index where maxVal currently exists in the array.

Step 7: Swap the elements

Swap the element at the current position with the element at the position of maxVal .
This places the largest possible number at the current index.

Step 8: Update positions and swap count

Update the HashMap with the new indices after the swap.

Decrease the value of k (one swap used) and decrease maxVal for the next iteration.

Step 9: Continue until swaps are used

Repeat the process until either all positions are checked or the allowed number of swaps k becomes zero.

Step 10: Return the final permutation

After performing the swaps, return the modified array which represents the largest possible permutation.

Complete JAVA Code:

```
import java.util.*; import java.io.*; public class
Solution {    static int[] largestPermutation(int
k, int[] arr) {        int n = arr.length;

        // Map each value to its current index for O(1) lookup
Map<Integer, Integer> pos = new HashMap<>();

        for (int i = 0; i < n; i++)

        {
pos.put(arr[i], i);

        }

        int maxVal = n; // largest possible value in array
for (int i = 0; i < n && k > 0; i++) {

        // If current element is already the max possible,
skip        if (arr[i] == maxVal) {            maxVal--;
continue;

        }

        // Find where maxVal currently is and swap it to position
i        int j = pos.get(maxVal);        // Update position map
pos.put(arr[i], j);

        pos.put(maxVal, i);
```

```

        // Swap in
        array      arr[j] =
arr[i];          arr[i] =
maxVal;
maxVal--;        k--;
    }    return arr;
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));    StringTokenizer st = new
StringTokenizer(br.readLine());    int n = Integer.parseInt(st.nextToken());
int k = Integer.parseInt(st.nextToken());    int[] arr = new int[n];    st = new
StringTokenizer(br.readLine());    for (int i = 0; i < n; i++) {        arr[i] =
Integer.parseInt(st.nextToken());
    }

    int[] result = largestPermutation(k,
arr);    StringBuilder sb = new
StringBuilder();    for (int i = 0; i < n;
i++) {        if (i > 0) sb.append(" ");
sb.append(result[i]);
    }

    System.out.println(sb.toString());
    }
}

```

Observation/Solution:

In this program, the goal is to arrange the numbers in the array to form the largest possible permutation using at most k swaps. First, the program reads the array and stores the position of each element using a HashMap so that the location of any value can be found quickly.

Then the program goes through the array from the first position. For each position, it checks whether the largest possible value for that position is already present. If it is not, the program finds where that largest value is located and swaps it with the current element. After performing the swap, the positions in the map are updated and the number of remaining swaps is reduced.

This process continues until there are no swaps left or all elements are checked. By always placing the largest available number at the beginning positions, the program successfully creates the largest possible permutation of the array.